

SKILLCLAW

Collective Skill Evolution for Multi-User LLM Agent Ecosystems

Let Skills Evolve Collectively with Agentic Evolver

arxiv.org/abs/2604.08377



Agent Skills Remain Static After Deployment

Users Rediscover Solutions Independently

STATIC SKILLS (Today)

- Skills manually installed from hub
- No learning from runtime experience
- Failures & recoveries repeat across users
- Memory-based methods: instance-specific, hard to generalize
- Knowledge never accumulates at system level



EVOLVING SKILLS (SkillClaw)

- Skills evolve from runtime interaction
- Cross-user trajectories aggregated
- Agentic evolver reasons over evidence
- Verified updates synchronized to all agents
- Shared ecosystem improves continuously

Closed-Loop Pipeline: Multi-User Interaction to Shared Skill Evolution



Agentic Evolver: Three-Stage Pipeline

Open-ended agent reasoning, not predefined update rules

1

Evidence

Analyze session trajectories preserving full action–feedback causal chains to identify behavioral patterns across users

2

Attribution

Diagnose root causes of failures and successes, separating generalizable improvements from idiosyncratic fixes

3

Evolution

Propose skill mutations — refining existing skills or creating new ones — then verify before merging into the shared repository

Cross-User Trajectory Aggregation

Exposing Skill Behavioral Boundaries

A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix.

Compatible Claw-Style Agent Systems

OpenClaw

CoPaw

IronClaw

PicoClaw

ZeroClaw

NanoClaw

NemoClaw

Different users exercising the same skill under diverse contexts produce complementary views of that skill's behavioral boundary, revealing both conditions under which it works and those under which it breaks.

Case Study: Multimodal Creative Pipeline

6-Day Evolution Log — 1 of 6 Accepted (Conservative Verification)

Day	Candidate Skill	Outcome	Rationale
1	validate-tmp-workspace-inputs	Accepted	Check /tmp_workspace inputs & env setup
2	multimodal-input-validation-and-setup	Rejected	Multimodal validation & output init — no improvement
3	multimodal-creative-task-pipeline	Rejected	Creative pipeline (PDF/video/image) — no improvement
4	multimodal-creative-task-pipeline (impr.)	Rejected	Added image classification & validation
5	pipeline + validate-required-input-files	Rejected	Fail-fast per-file validation — no improvement
6	multimodal-creative-task-pipeline (cand.)	Rejected	Extended PDF-to-poster paths — not admitted

Conservative verification prevents regressions — only genuinely improving candidates pass

Case Study: Git Push Auth-Fallback Skill

6-Day Evolution Log — 3 of 6 Accepted (Iterative Refinement)

Day	Candidate Skill	Outcome	Rationale
1	git-push-with-auth-fallback	Accepted	Safe fallback instead of blocking on push failure
2	git-push-with-auth-fallback	Accepted	Unified patch/bundle filenames, reduced inconsistency
3	auth-fallback + git-clone-to-directory	Accepted	Auth-alternative paths & secrets audit; fixed subshell pitfalls
4	(retest)	Accepted	Same-pool retest; validator confirmed current pool as best
5	git-push-with-auth-fallback	Rejected	"Push hang = auth failure" — no improvement over pool
6	git-push-with-auth-fallback	Rejected	Identity config & filename consistency — not admitted

Iterative refinement works when evidence supports it — 3 accepted improvements before plateauing

Three Properties That Distinguish SkillClaw

01

Collective Evolution

Knowledge from individual interactions contributes to a shared, continuously improving skill ecosystem. No single user bears the burden of improvement alone.

02

Fully Automatic

Skill evolution is driven by runtime interaction without manual curation or explicit user intervention. The system improves as it is used.

03

Agentic Evolution Paradigm

Skill updates are produced through open-ended reasoning over evidence — not predefined rules. The evolver is itself an autonomous agent.

Evaluated on WildClawBench with Qwen3-Max — SkillClaw points toward continuously improving agent systems where skills are living artifacts, not static resources.